



Introduction to NLP

AIAA 4051 Lecture 5

Instructor: Sihong Xie

Email: sihongxie@hkust-gz.edu.cn

AI Thrust, HKUST(GZ)

HMM prediction

- Want to find the best sequence of POS tags for a given sentence.
- Vocabulary V
- Set of N POS tags $S = \{s_1, \dots, s_n\}$

- Observed sentence $O = [o_1, \dots, o_T], o_t \in V$

All we gotta do is go around the corner

- Hidden states: $Q = [q_1, \dots, q_T], q_t \in S$

q_1 q_2 q_9

- MAP (maximum a posterior) prediction given parameters $\theta = (A, B, \pi)$

$$Q^* = \arg \max_Q \Pr(Q|O; \theta)$$

DT PRP VBN VB VBZ VB IN DT NN

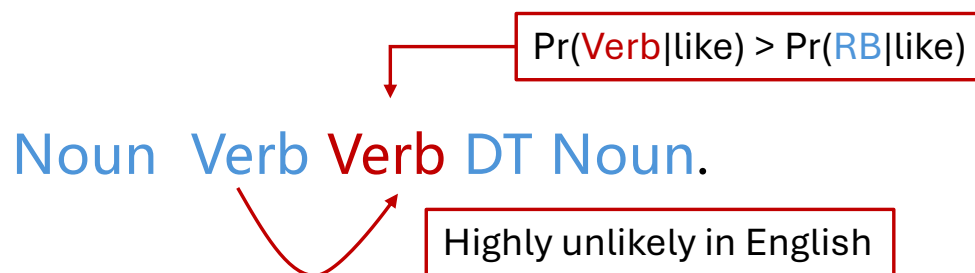
- Difficulty: there are exponentially many possible sequences.

A running example

- POS tagging the following sentence

"Time flies like an arrow."

- If we tag the tokens one-by-one independently, using the most frequent tag of each word, we have



- The "local" Pr(Verb | like) does not consider the neighbors' output
 - but the neighbors' tags also need to be predicted according to the tag of "*like*".
 - "egg-first-chicken-first" question.

Structured prediction

- We should making predictions of all items at the same time.
 - Evaluate the quality of the predictions as a whole.
- This is called “**structured prediction**”.
 - Also useful for computer vision problems.
 - The process of making structured predictions is called “**decoding**”.

- Formulation

- Rather than $\hat{q}_i = \arg \max_{q_i \in S} P(q_i | o_i; \theta) \quad \forall i = 1, \dots, T$

- We make joint prediction of the whole sequence of tags

$$Q^* = \arg \max_Q \Pr(Q | O; \theta) \quad Q = [q_1, \dots, q_T], q_t \in S$$

- The optimal predicted sequence considers both
 - emission probability of the whole observed sentence
 - transition probability of the whole hidden sequence of POS tags.
- But the exhaustive search for the optimal sequence if of **$O(|S|^T)$ time complexity**.

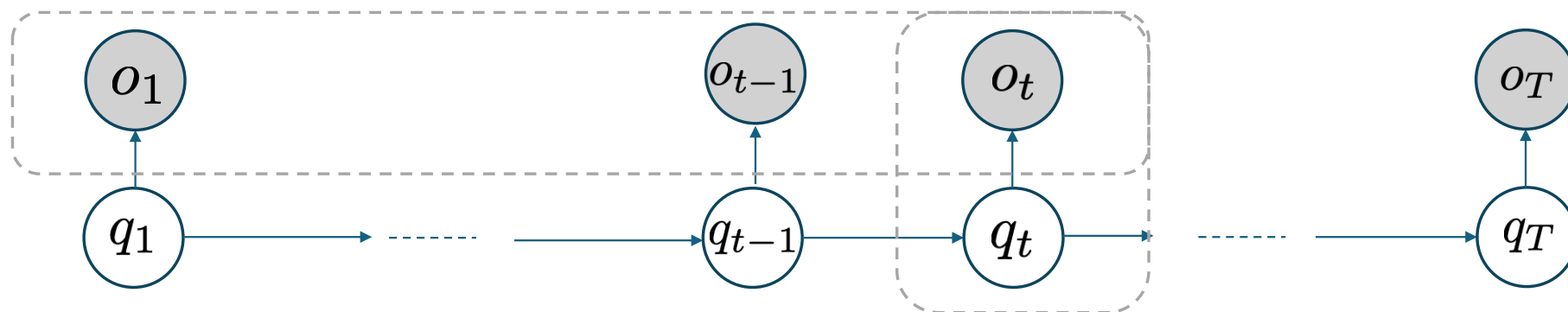
Viterbi algorithm

- To solve the structured prediction of POS tagging efficiently, we exploit the special structure of the tag sequence.
 - It is a chain to enable a forward algorithm-like dynamic programming (DP).
 - We are maximizing over all sequences, but the step-wise nature leads to **optimal structures** and **overlapping sub-problems**, making DP suitable.

- Define the probability

$$v_t(j) = \max_{q_1, \dots, q_{t-1}} \Pr(q_1, \dots, q_{t-1}, q_t = j, o_1, \dots, o_t)$$

- The maximum probability of seeing POS-tag j and the observed words up to o_t .



Viterbi algorithm

- Base case when $t = 1$

$$v_1(j) = \max_{\emptyset} \Pr(q_1 = j, o_1)$$

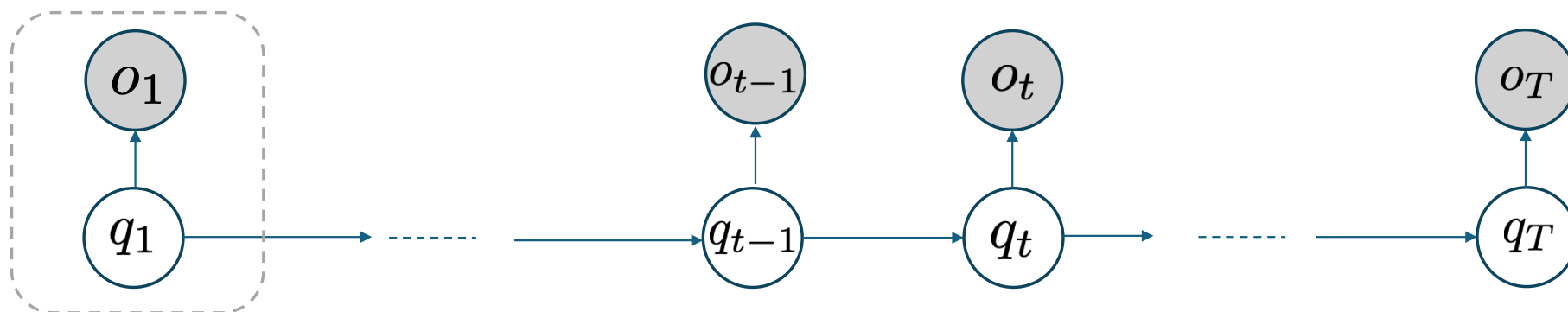
(by definition)

$$= \Pr(q_1 = j, o_1)$$

(only one value to pick)

$$= \Pr(q_1 = j) \Pr(o_1 | q_1 = j)$$

(decomposition according to HMM)



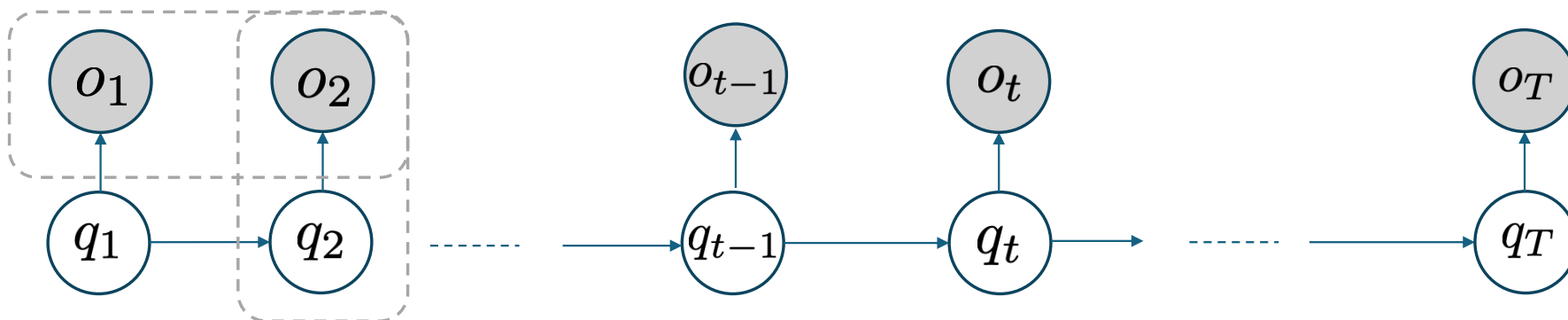
Viterbi algorithm

- The case when $t = 2$

$$v_2(j) = \max_{q_1} \Pr(q_1, q_2 = j, o_1, o_2) \quad (\text{by definition})$$

$$= \max_{q_1} \Pr(q_1) \Pr(o_1|q_1) \Pr(q_2 = j|q_1) \Pr(o_2|q_2 = j) \quad (\text{decomposition according to HMM})$$

$$= \max_{q_1} v_1(q_1) \Pr(q_2 = j|q_1) \Pr(o_2|q_2 = j) \quad (\text{reuse cached solution})$$



Viterbi algorithm

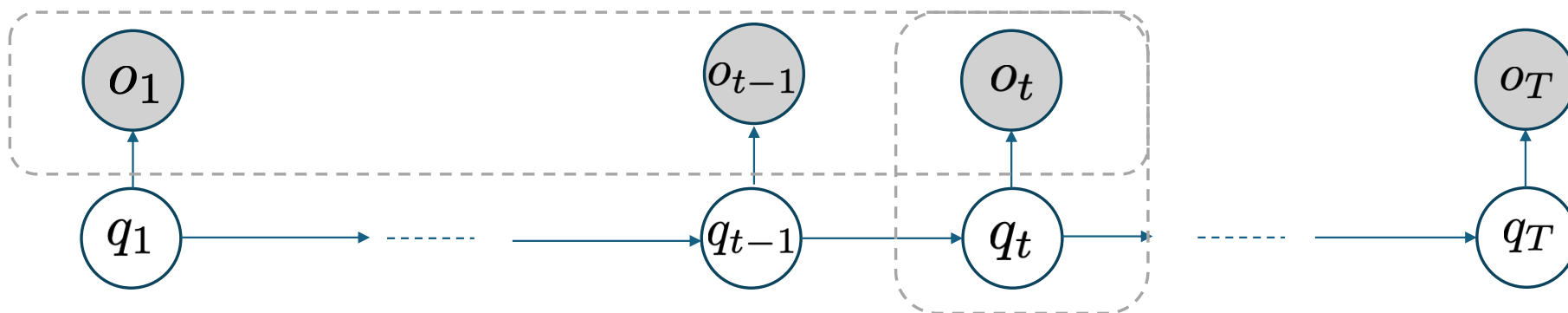
- The general case

$$v_t(j) = \max_{q_1, \dots, q_{t-1}} \Pr(q_1, \dots, q_{t-1}, q_t = j, o_1, \dots, o_t) \quad (\text{by definition})$$

$$= \max_{q_{t-1}} \max_{q_1, \dots, q_{t-2}} \Pr(q_t = j | q_{t-1}) \Pr(o_t | q_t = j) \Pr(q_1, \dots, q_{t-2}, q_{t-1}, o_1, \dots, o_{t-1}) \quad (\text{HMM decomposition})$$

$$= \max_{q_{t-1}} \Pr(q_t = j | q_{t-1}) \Pr(o_t | q_t = j) \max_{q_1, \dots, q_{t-2}} \Pr(q_1, \dots, q_{t-2}, q_{t-1}, o_1, \dots, o_{t-1}) \quad (\text{exchange of max's})$$

$$= \max_{q_{t-1}} \Pr(q_t = j | q_{t-1}) \Pr(o_t | q_t = j) v_{t-1}(q_{t-1}) \quad (\text{reuse cached solution})$$



Viterbi algorithm

- Recording the optimal path
 - Pointer to what leads to the optimal path at each step

$$p_t(j) = \arg \max_k v_{t-1}(k) a_{kj} b_j(o_t)$$

- Useful for back-tracking to re-construct the optimal path.

$$Q^* = \arg \max_Q \Pr(Q|O; \theta)$$

- Otherwise, we only have the optimal value, but don't know which path has the optimal value.
- Start with $q_T^* = \max_k v_T(k)$
- Backtracking $q_{t-1}^* = p_t(q_t^*)$

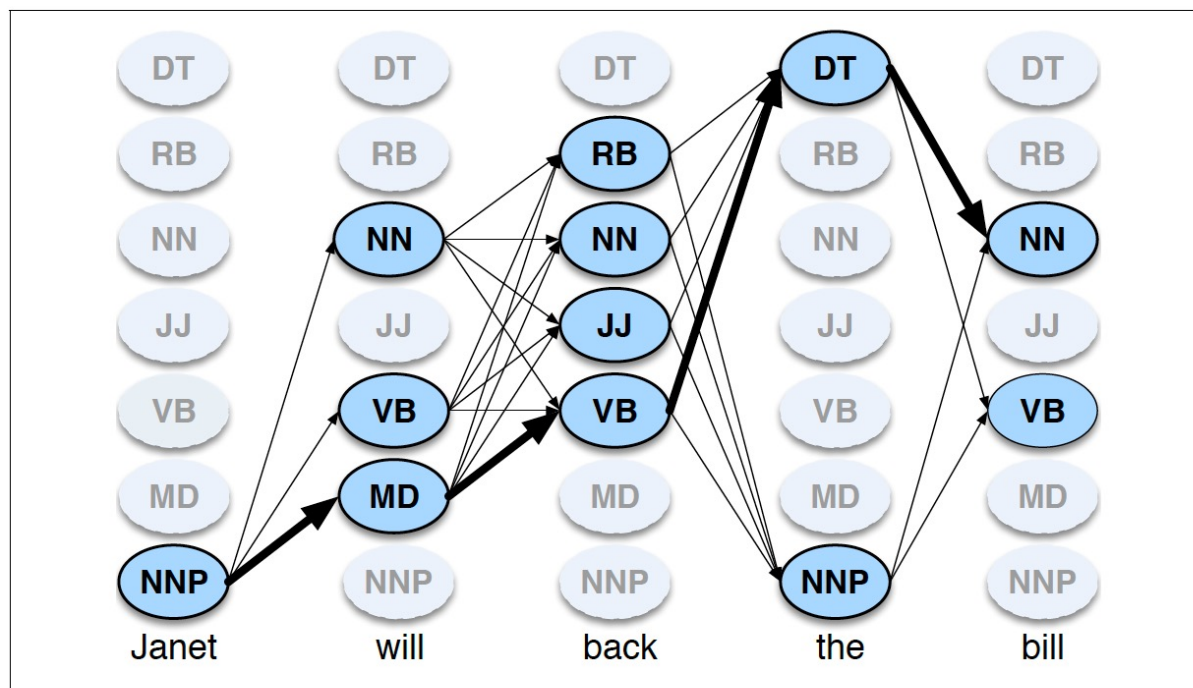


Figure 17.11 A sketch of the lattice for *Janet will back the bill*, showing the possible tags (q_i) for each word and highlighting the path corresponding to the correct tag sequence through the hidden states. States (parts of speech) which have a zero probability of generating a particular word according to the B matrix (such as the probability that a determiner DT will be realized as *Janet*) are greyed out.

Image source: Figure 17.11 of SLP3 (2026 version)

Viterbi algorithm

Compute maximum probability and the optimal sequence (decoding)

1. Initialize $v_1(i)$ for each value i of the first hidden state q_1 .
2. for $t = 2, \dots, T$
 for $j = 1, \dots, N$
 compute $v_t(j) = \max_k v_{t-1}(k) a_{kj} b_j(o_t)$
 record back-pointers $p_t(j) = \arg \max_k v_{t-1}(k) a_{kj} b_j(o_t)$
3. Backtracking to find Q^*
4. Return $\Pr(Q^*|O) = \max_j v_T(j)$

Running example

We don't calculate

π	Starting tag	NNP (Pro-Noun)	N (Noun)	
		0.5	0.5	
A	Current ↓ \ Next →	NNP (Pro-Noun)	N (Noun)	V (Verb)
	NP (Pro-Noun)	0.2	0.1	0.7
	N (Noun)		0.001	0.7
	V (Verb)			

B	Q ↓ \ O →	They	base
	NNP (Pro-Noun)	0.8	0
	N (Noun)	0	0.6
	V (Verb)	0	0.4

We don't need the entries in the following calculation when they are empty.

They The max is trivial as base

$v_1(N) = v_1(V) = 0$

$$\begin{aligned}
 v_1(NNP) &= \Pr(NNP) \Pr(They|NNP) \\
 v_1(N) &= \Pr(N) \Pr(They|N) = 0 \\
 v_1(V) &= \Pr(V) \Pr(They|V) = 0
 \end{aligned}$$

$v_2(V) = \max\{ v_1(NNP) \Pr(V|NNP) \Pr(base|V),$
 $v_1(N) \Pr(V|N) \Pr(base|V),$
 $v_1(V) \Pr(V|V) \Pr(base|V) \}$

$v_2(N) = \max\{ v_1(NNP) \Pr(N|NNP) \Pr(base|N),$
 $v_1(N) \Pr(N|N) \Pr(base|N),$
 $v_1(V) \Pr(N|V) \Pr(base|N) \}$

$v_2(NNP)$ can be calculated similarly.

V and N compete regarding transition and emission probabilities.

Fun facts about Viterbi

- Viterbi is a researcher who later co-founded Qualcomm. He graduated from USC and later donated gifts to the Viterbi School of Engineering at USC.
- The Viterbi algorithm was first invented in 1967 for communication.
- *"Through his invention of the celebrated Viterbi algorithm in communications, which has been used in billions of cell phones, space communications, voice recognition, data recording, search, and DNA sequencing".*



Estimating HMM parameters

- Given: m observed sentences, each looks like $O = [o_1, \dots, o_T], o_t \in V$ (lengths vary).
- Find: HMM $\lambda = [A, B, \pi]$
- The problem is easy if the hidden states $Q = [q_1, \dots, q_T], q_t \in S$ are known. This is called “supervised learning”. Using MLE, we have

$$a_{ij} = \frac{C(i \rightarrow j)}{C(i)} \quad b_{i,o} = \frac{C(i \rightarrow o)}{C(i)} \quad \pi_i = \frac{C(q_1 = i)}{m}$$

- C is the counting function, counting how many times each event occurred in the training corpus. For example, $C(i \rightarrow j)$ is the number of times POS tag i transits to tag j .
- m is the number of sentences in the training corpus.

Expectation-Maximization (EM) algorithm

- EM algorithm is commonly used in ML/AI for estimating model parameter with unlabeled data.
 - Unsupervised or semi-supervised learning.
- The E-step: calculate estimated probabilities of labels on the unlabeled data.
 - The estimated probabilities are called “pseudo labels”.
 - This is labeling the data with “soft-labels” (as the model is imperfect and uncertain about data labels).
- The M-step: estimate model parameters using pseudo labels as in MLE.
- Then go back to the E-step using updated parameters from the M-step.

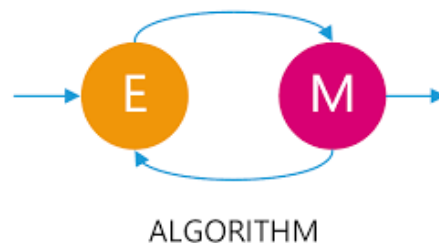
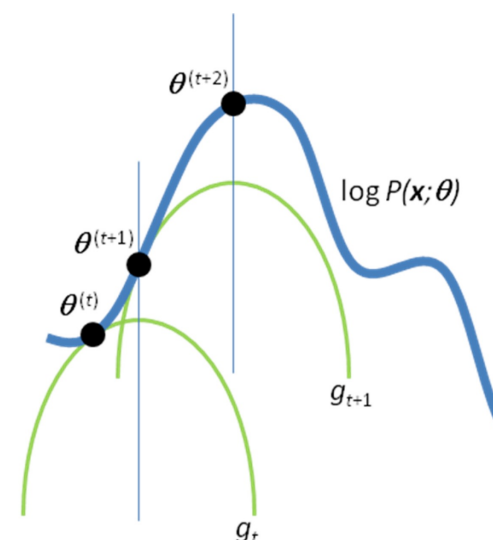
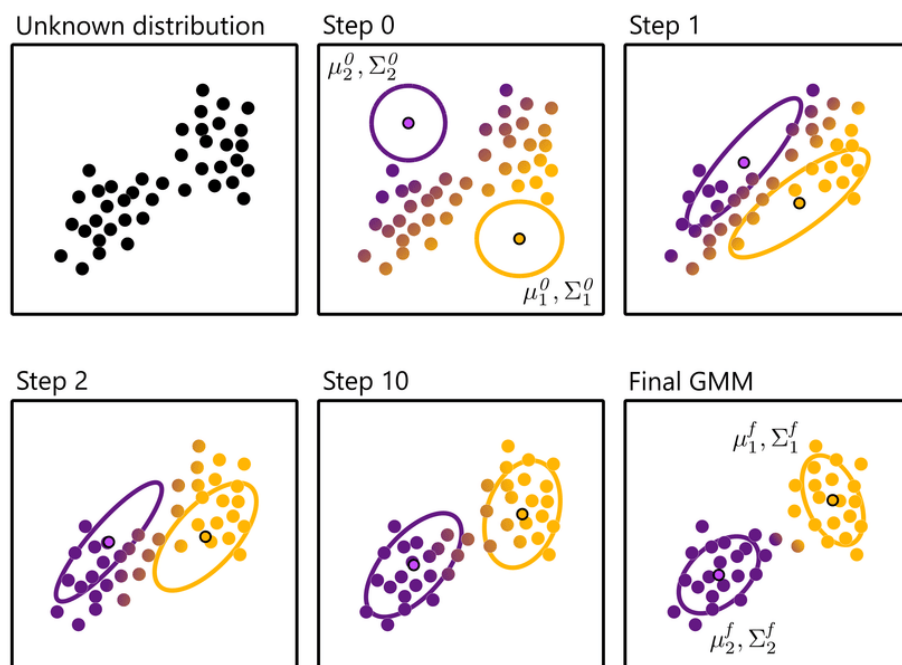


Image source: <https://www.indowhiz.com/articles/en/the-simple-concept-of-expectation-maximization-em-algorithm/>

Expectation-Maximization (EM) algorithm

- Estimating means and covariance from unlabeled data.
- Each cycle of EM guarantees increasing likelihood of data.



Supplementary Figure 1 Convergence of the EM algorithm. Starting from initial parameters $\theta^{(t)}$, the E-step of the EM algorithm constructs a function g_t that lower-bounds the objective function $\log P(x; \theta)$. In the M-step, $\theta^{(t+1)}$ is computed as the maximum of g_t . In the next E-step, a new lower-bound g_{t+1} is constructed; maximization of g_{t+1} in the next M-step gives $\theta^{(t+2)}$, etc.

https://www.researchgate.net/figure/illustrative-example-of-the-EM-algorithm-for-estimation-of-parameters-of-a-GMM-with-2_fig19_329773081
https://seunghan96.github.io/stat/9.-em-EM_algorithm%281%29/

Estimating HMM parameters

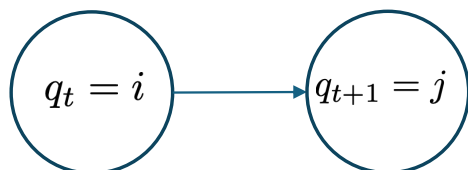
Estimating transition matrix A :

- We don't know the POS tag sequences definitely and $a_{ij} = \frac{C(i \rightarrow j)}{C(i)}$ is not available.
- Rather, we need to find "soft" counts, using probabilities estimated based on $\lambda = [A, B, \pi]$
- Find probability of co-occurrence of two tags (i, j) at location t :

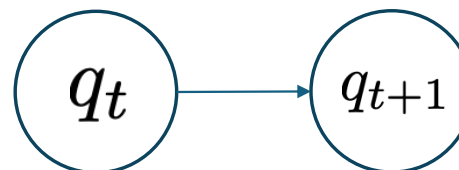
$$a_{ij} = \frac{\sum_{k=1}^m \sum_{t=1}^T \xi_t^{(m)}(i, j)}{\sum_j \sum_{k=1}^m \sum_{t=1}^T \xi_t^{(m)}(i, j)}$$

- Soft-label of transition from i transits to j $\xi_t(i, j) = \Pr(q_t = i, q_{t+1} = j | O, \lambda)$

Hard labels:



Soft labels:

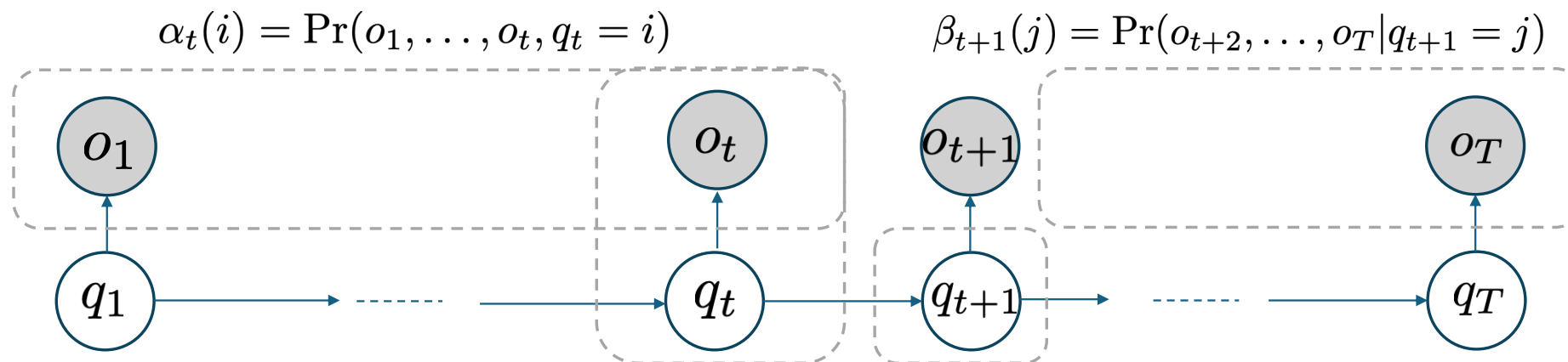


Estimating HMM parameters

- Estimating the soft label

$$\xi_t(i, j) = \Pr(q_t = i, q_{t+1} = j | O, \lambda) = \Pr(q_t = i, q_{t+1} = j, O | \lambda) / \Pr(O | \lambda) \quad (\text{by definition})$$

$$= \alpha_t(i) \Pr(q_{t+1} = j | q_t = i, \lambda) \Pr(o_{t+1} | q_{t+1}) \beta_{t+1}(j) / \Pr(O | \lambda) \quad (\text{by HMM assumption})$$



Estimating HMM parameters

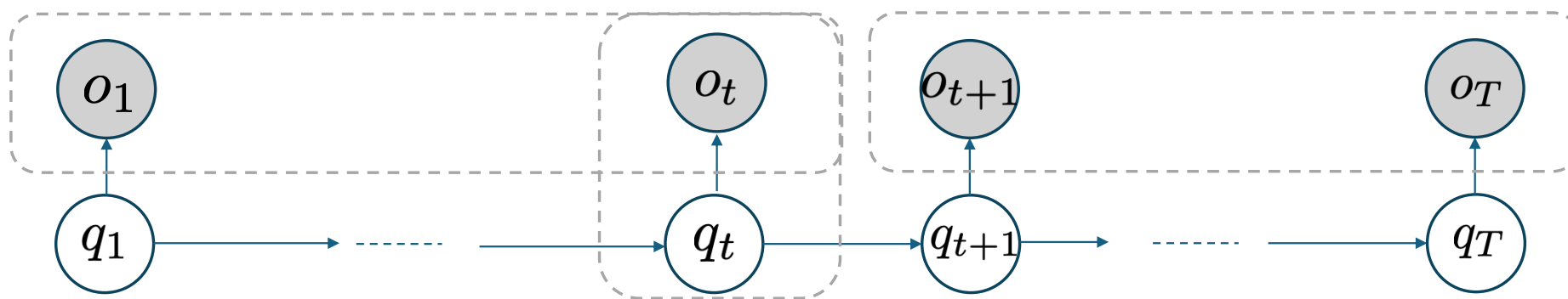
Estimating B :

- Similarly, $b_{i,o} = \frac{C(i \rightarrow o)}{C(i)}$ is not available.
- Find probability of co-occurrence of tag i and word o at location t :
 - find probability $\gamma_t(i) = \Pr(q_t = i | O, \lambda)$

$$= \Pr(q_t = i, O | \lambda) / \Pr(O | \lambda)$$

$$= \alpha_t(i) \beta_t(i) / \Pr(O | \lambda)$$

$$\alpha_t(i) = \Pr(o_1, \dots, o_t, q_t = i) \quad \beta_t(i) = \Pr(o_{t+1}, \dots, o_T | q_t = i)$$



$$b_{io} = \frac{\sum_{k=1}^m \sum_{t=1, o_t=o}^T \gamma_t^{(m)}(i)}{\sum_{k=1}^m \sum_{t=1}^T \gamma_t^{(m)}(i)}$$

Estimating HMM parameters

EM (expectation-maximization) algorithm for HMM estimation (Baum-Welch Algorithm):

Given: m observed sentences, some possibly labeled (semi-supervised learning)

Initialize $\lambda = [A, B, \pi]$ either randomly or using the POS-tags if available.

loop until convergence

E-step: run forward and backward algorithms to find

$$\alpha_t(j) = P(o_1, \dots, o_t, q_t = j) \quad \beta_t(i) = P(o_{t+1}, \dots, o_T | q_t = i)$$

M-step: find

$$\xi_t(i, j) = \Pr(q_t = i, q_{t+1} = j | O, \lambda)$$

$$\gamma_t(i) = \Pr(q_t = i | O, \lambda)$$

Caution: EM only finds local optima and a bad initialization of parameters can lead to bad results. Some supervisions are helpful.

$$a_{ij} = \frac{\sum_{k=1}^m \sum_{t=1}^T \xi_t^{(m)}(i, j)}{\sum_i \sum_{k=1}^m \sum_{t=1}^T \xi_t^{(m)}(i, j)}$$

$$b_{io} = \frac{\sum_{k=1}^m \sum_{t=1, o_t=o}^T \gamma_t^{(m)}(i)}{\sum_{k=1}^m \sum_{t=1}^T \gamma_t^{(m)}(i)}$$

starting probabilities are left as an exercise

| Extra readings

- For structured prediction, see the textbook "Advanced Structured Prediction" by Sebastian Nowozin, etc..
- For EM algorithms, please refer to fundamental ML books such as Pattern Recognition and Machine Learning and DSAA 2011 slides.
- We will touch more decoding algorithms, such as beam search, later.

| Demon

- See demo for lecture 4 that includes the Viterbi algorithm.

Conclusion

- Viterbi decoding algorithm, a dynamic programming algorithm that find the optimal POS tag sequence, rather than predicting tags of the words independently.
- EM algorithm for HMM parameter estimation: unsupervised for jointly predicting tag distribution and parameter estimation.
- Next: syntax and syntactic parsing.
- Let us know your thoughts using the QR code on Canvas

Quiz

1. Pen and paper; no compute or cellphone allowed.
 2. Turn in your answer sheet when you leave the classroom.
- Q1 (T or F): Viterbi algorithm is a dynamic programming algorithm with time complexity $O(TN)$, for a sentence of length T and N possible POS tags.
 - A1: F. The correct answer is $O(TN^2)$
 - Q2 (short answer): Write down the defining equation of $v_t(j)$ in Viterbi algorithm.
 - A2: $v_t(j) = \max_k v_{t-1}(k) a_{kj} b_j(o_t)$ or $v_t(j) = \max_{q_1 \dots q_{t-1}} \Pr(q_1, \dots, q_{t-1} q_t = j, o_1, \dots, o_t)$
 - Q3 (multiple choice): What algorithm/principles are used in the EM algorithm for estimating HMM parameters?
 - A: dynamic programming
 - B: greedy algorithm
 - C: MLE
 - D: gradient descent